

UNDER THE HOOD

No black boxes. Every formula, constant and score on this page is real code driving the robot right now — most of it in one readable Python file: `tb4_claude_nav.py`.

1 The cost map

where it's safe to drive — with no prebuilt map

Each lidar sweep fills a **120×120 grid** (5 cm/cell = 6×6 m, robot at centre); an OpenCV **L2 distance transform** gives each cell its clearance **d**, then:

$$\text{cost}(d) = \begin{cases} \infty & d \leq 0.18 \text{ m} \\ 1.5 \cdot e^{(-8 \cdot (d - 0.18))} & 0.18 < d < 0.30 \\ 0 & d \geq 0.30 \text{ m} \end{cases}$$

A soft **"don't hug walls"** band ($\approx$ Nav2's `cost_scaling_factor`): equal-length routes aren't tied — more clearance wins.

`build_grid() - cv2.distanceTransform(free, DIST_L2)`

2 Frames & rotation matrices

why the map isn't sideways

The lidar is mounted **+90° from the robot frame**, so every beam (r, θ) is rotated into `base_link` with a 2-D rotation matrix:

$$\begin{bmatrix} x_b \\ y_b \end{bmatrix} = \begin{bmatrix} \cos \phi & -\sin \phi \\ \sin \phi & \cos \phi \end{bmatrix} \begin{bmatrix} x_l \\ y_l \end{bmatrix} \quad \phi = +\pi/2$$

beam (r, θ): $x = -r \cdot \sin \theta, \quad y = r \cdot \cos \theta$

Heading comes from the odometry **quaternion**: $\psi = \text{atan2}(2(wz+xy), 1-2(y^2+z^2))$. The code's own comment:

"without this the whole grid is 90° off reality."

`build_grid(), _quat_to_yaw() - TF = +1.571 rad`

3 A* search

the classic pathfinder, ~35 lines, on a live map

```
heap = [(h(start), 0, start)] # f = g + h
while heap:
    g, (r,c) = heappop(heap)
    for (dr,dc,step) in MOVES: # 1 or √2
        g2 = g + step + cost[cell]
        if g2 < best[cell]: heappush(...)
```

8-connected, h = Euclidean; the soft penalty rides in **g**, so paths prefer open space. **Line-of-sight** smoothing trims corners — capped so shortcuts can't cut the wall-hug band.

`astar() - 3 Hz replan, pure pursuit on 0.5 m lookahead`

4 Natural language → program

any command → ranked pick from the behavior library

"push the red ball to the wall" ranks the `.toy` program library in two stages:

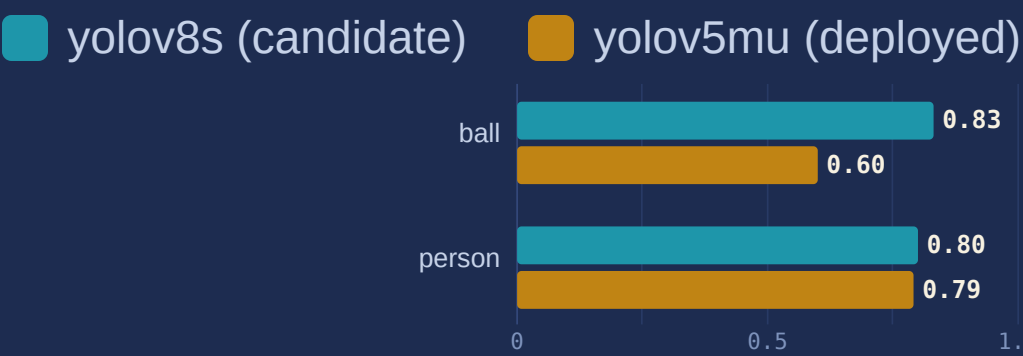
```
# 1. verbatim trigger (longest wins) → 1.0
"push the ball" ∈ text → push_ball_to_wall
# 2. else token overlap, query Q vs corpus W
score = |Q ∩ W| / √(|Q| + 1)
```

Stage 1 keeps a big word pool from shadowing an exact trigger. Deliberately **dependency-free** — the code marks the seam: **"swap in an embedding model here."**

`match_program() - colour words ("red") filter detections`

5 Evaluating the detector

is the new model actually better? measure, don't guess



Offline AP per class — the ball gap is why the swap is staged.

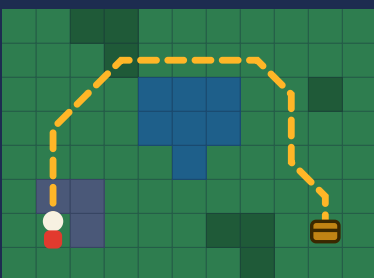
Deploy: 416 input, conf 0.3, IoU 0.5, 6-SHAVE RVC2 blob, ~6 Hz on the camera VPU. **Lesson:** a mis-compiled decode head once scored a bogus ~15% — **eval the deployed artifact**, not just the `.pt`.

`models/YOLOV8S_SWAP.md · training/ · objdet18/`

6 The hacks that make it work

sensor fusion & the duct tape that survives a demo

- ▶ **Depth that ignores the background** — distance = the **25th percentile** of stereo depth in the box: the spatial NN kept reporting the wall behind the ball.
- ▶ **Two sensors, one answer** — pixel → bearing, then distance = **min(stereo, lidar at that bearing)**: camera says **what**, the nearer sensor says **where**.
- ▶ **Detections become obstacles** — each seen object stamps a **no-go disk** into the cost map — except the push target, whose cost is **cleared** so A* may hit it.
- ▶ **Stalling is the success signal** — a push is done when odometry stops moving: the stuck detector doubles as **"ball pinned"**.
- ▶ **The alarm that's ignored** — bump / cliff / wheel-drop e-stop instantly; **backup_limit** cries wolf, so it's deliberately dropped.
- ▶ **Sensors that heal themselves** — docking stops the lidar motor, USB2 stalls the camera; stale timestamps trigger **auto-restarts**.



Same A*, different worlds — the takeaway you can use tomorrow

Panel 3's A* also routes **RPG characters** — tiles are cells, terrain is cost: grass **1**, forest **2**, water ∞ — the same loop behind **GPS routing** and **warehouse robots**. `astar()` never knows whether the world is real.